

Compte Rendu

SAE – Détection de biomes sur des exoplanètes

Nathan Späth – Matthieu Cabot – Arthur Jaquet – Enzo Marchal – Julien Vincent

Cette SAE avait pour objectif de détecter et mettre en lumière les différents biomes présents sur une exoplanète en analysant la couleur des pixels de celle-ci. Nous devions par la suite définir les différents écosystèmes présents dans chaque biome.

Le compte rendu sera divisé en 3 grandes parties, la répartition du travail et les algorithmes, les tests effectués pour finir par présenter les résultats.

Choix des algorithmes :

Pour effectuer cette série de tests, nous avons d'abord cherché quel algorithme, parmi ceux mis à notre disposition, était le plus optimisé pour, tout d'abord, générer une palette de couleurs à partir d'une image, puis pour trouver et afficher les écosystèmes.

Pour la première étape, nous avons choisi d'utiliser K-means. Cet algorithme nous permet de choisir dès le départ le nombre de couleurs que nous voulons dans la palette, car K-means fonctionne avec un paramètre K qui nous permet de limiter la division de l'image. Nous avons écarté DBSCAN et HAC car, ne pouvant pas limiter le nombre de divisions de l'image, ils ne permettent pas d'obtenir un nombre de couleurs précis en sortie.

Pour ce qui est de l'algorithme permettant de trouver les écosystèmes dans un biome, nous avons choisi de partir sur DBSCAN. Ce qui était un désavantage pour créer une palette de couleurs devient un atout pour trouver des écosystèmes, car DBSCAN cherche simplement les pixels voisins selon un rayon défini en s'étendant de proche en proche. Il permet aussi de limiter l'apparition de bruit : si un pixel n'a pas le nombre de voisins nécessaire pour appartenir à un écosystème, il ne sera compté dans aucun. À l'inverse, l'utilisation de K-means serait beaucoup plus limitée ici, car il faudrait savoir à l'avance, pour chaque biome, quel est le nombre exact d'écosystèmes présents pour avoir un résultat cohérent, ce qui n'est pas le cas.

Nous avons aussi essayé d'implémenter HAC pour trouver les écosystèmes, mais à cause de son temps d'exécution (complexité de N^3), nous n'avons pas pu réaliser suffisamment de tests pour l'analyser convenablement.

Ainsi, l'ensemble de nos tests est basé sur la modification des métriques de K-means et DBSCAN pour voir quelle combinaison permet d'avoir les résultats les plus satisfaisants le plus rapidement possible.

Tests réalisés :

Afin de déterminer la configuration optimale pour notre programme, nous avons développé un environnement de test (MainRecherche.java). Cet outil nous a permis d'exécuter une série de test sur nos images en effectuant en modifiant plusieurs paramètres :

L'intensité du lissage : Test du rayon du filtre de flou (FlouRayon à 3 et 5).

La fragmentation des couleurs : Test du nombre de centroïdes pour l'algorithme K-Means (K à 10, 15, 20).

Les contraintes géométriques de DBSCAN :

Test du rayon de voisinage Eps (3.0 et 5.0).

Test de la densité minimale MinPts (20 et 50).

Chaque combinaison génèrait une extraction de biomes et un regroupement en écosystèmes. Le programme mesurait le temps d'exécution global en millisecondes et enregistrait les résultats dans un fichier CSV (resultats_recherche.csv) pour analyse algorithmique.

Résultats obtenus :

L'analyse de nos phases de tests a mis en évidence certaines limites de l'algorithme DBScan (qui possède une complexité de N^2) :

Le piège des zones uniformes (Impact du Flou et de K-Means) : Nous avons constaté qu'un flou trop élevé (rayon de 5) ou un K trop faible ($K=10$) crée des biomes gigantesques et ininterrompus (ex: un Océan de 300 000 pixels). À cause de sa complexité, DBScan tente alors d'effectuer des dizaines de milliards d'opérations, faisant passer le temps d'exécution de quelques secondes à plus de 2 heures. À l'inverse, un $K=20$ fragmente l'image intelligemment et divise le temps de calcul par 3.

L'impact de la distance de recherche (Eps) : Passer un rayon de 3.0 à 5.0 double systématiquement le temps de calcul, car la fonction de recherche de voisinage (regionQuery) trouve beaucoup trop de points à analyser et sature la mémoire.

L'impact de la densité (MinPts) : Augmenter le nombre de points minimum accélère drastiquement l'algorithme. Si un pixel n'a pas assez de voisins, DBScan coupe court à la recherche et le classe comme "bruit", évitant des calculs d'expansion inutiles. Cependant, un MinPts trop élevé par rapport à Eps (ex: demander 50 pixels dans un rayon de 3.0) rend la création d'écosystèmes impossible (aucun écosystème trouvé).

Conclusion :

En conclusion, l'algorithme DBScan est très bon pour créer des écosystèmes à partir de biomes mais a beaucoup de difficulté lorsque le biome devient très grand. Notre main de test intègre désormais un système de jeu de paramètres. Si un biome dépasse 75 000 pixels, l'algorithme abandonne le mode normal pour projeter les données sur une grille de résolution réduite (Modes Rapide ou Très Rapide).

Grâce à cette architecture et à nos tests, nous avons pu définir un point d'équilibre entre les 2 algorithmes :

- Lissage : Flou de rayon 3 et K-Means avec K=20 (pour pré-segmenter sans dénaturer).
- Clustering : DBSCAN avec Eps = 4.0 et MinPts = 15.

Cette configuration nous permet de détecter des écosystèmes visuellement cohérents et pertinents en seulement quelques dizaines de secondes.